## Ejercicio 3: Tabla de Relaciones y Evidencias

Tabla comparativa indicando el tipo de relación formal entre cada par de clases y la evidencia exacta en el código:

| Par de clases | Tipo de relación | Evidencia (Línea de código / Método) |
|---|---|---|
| Estudiante y Persona | Herencia | class Estudiante extends Persona |
| Universidad y Carrera | Composición | carreras.add(new Carrera(nombreCarrera)) en crearCarrera() |
| Carrera y Docente | Agregación | public void agregarDocente(Docente docente) { docentes.add(docente); } |
| Materia y Horario | Composición | public Horario agregarHorario(...) { Horario s = new Horario(...); ... } |
| Docente y Materia | Asociación | private final List<Materia> materias; / materia.setDocente(this); |
| Inscripcion y Calificable | Realización (Interfaz) | class Inscripcion implements Calificable |
| Docente y Acta | Dependencia | public Acta generarActa(Materia m) { return new Acta(m); } |

## Ejercicio 4: Análisis de Relación entre Docente y Acta

**Explicación Teórica:** El Docente sólo mantiene una relación de Dependencia (uso temporal) con la clase Acta debido a que el objeto Acta no representa un atributo persistente ni forma parte del estado propio del Docente. El acta se crea, utiliza y retorna únicamente durante la ejecución del método generarActa(Materia materia).

**Cambio a Asociación:** Si la relación se transformara en una Asociación, la clase Docente tendría que almacenar internamente una colección o atributo permanente de las actas que genera a lo largo del tiempo.

**Código Propuesto:** Modificación del código si fuera Asociación:

```
class Docente extends Persona {
    private final String especialidad;
    private final List<Materia> materias = new ArrayList<>();
    // ASOCIACIÓN: El docente ahora guarda referencias permanentes de sus Actas
    private final List<Acta> actasGeneradas = new ArrayList<>();

    public Acta generarActa(Materia materia) {
```

```
        Acta acta = new Acta(materia);
        this.actasGeneradas.add(acta); // Se guarda la asociación permanente
        return acta;
    }

    public List<Acta> getActasGeneradas() {
        return Collections.unmodifiableList(actasGeneradas);
    }
}
```

## Ejercicio 5: Control de Partes (Todo - Parte)

**Relación Todo - Parte:** La diferencia fundamental entre crearCarrera() y agregarDocente() radica en quién controla la creación y el tiempo de vida del objeto parte:

**1. crearCarrera():** Representa una Composición estricta. La propia clase Universidad se encarga de instanciar internamente (new Carrera(...)) el objeto parte. La carrera no existe de forma independiente fuera de la Universidad. Si la Universidad deja de existir, sus carreras también son destruidas.

**2. agregarDocente():** Representa una Agregación. El objeto Docente es instanciado previamente de forma independiente en el main o en otro contexto. La clase Carrera solo recibe la referencia del docente existente y lo añade a su lista. Si la Carrera desaparece, el Docente sigue existiendo independientemente.

## Ejercicio 6: Corrección de Inconsistencias en el Modelo

Se muestran a continuación los 5 casos de inconsistencia detectados, especificando el código que lo provoca en el main, el resultado observado y la corrección propuesta utilizando únicamente estructuras de control básicas de POO (sin excepciones):

| Caso | Cómo provocarlo (main) | Resultado observado | Corrección propuesta (código sin excepciones) |
|---|---|---|---|
| Doble inscripción | `e1.inscribirse(poo);`<br>`e1.inscribirse(poo);` | El estudiante e1 queda registrado dos veces en la lista de inscripciones de la materia poo. | `En Estudiante.inscribirse:`<br>`for (Inscripcion i : inscripciones) {`<br>`  if (i.getMateria().equals(materia)) {`<br>`  System.out.println("Ya inscrito");`<br>`  return i;`<br>`  }`<br>`}` |
| Dos docentes en un curso | `d1.dictar(poo);`<br>`d2.dictar(poo);` | La materia cambia de docente a d2, pero poo permanece en la lista de materias de d1. | `En Materia.setDocente:`<br>`if (this.docente != null) {`<br>`  System.out.println("Materia ya asignada");`<br>`  return;`<br>`}` |

| | | | |
|---|---|---|---|
| | | | `this.docente = docente;` |
| Nota sin registrar | `Inscripcion i = e1.inscribirse(poo); i.calcularNota();` | Retorna 0.0 por defecto, lo que confunde una nota real de 0 con un examen no corregido. | `En Inscripcion:`<br>`private boolean notaRegistrada = false;`<br>`public double calcularNota() {`<br>`  if (!notaRegistrada) {`<br>`  System.out.println("Nota no registrada");`<br>`  return -1.0;`<br>`  }`<br>`  return nota;`<br>`}` |
| Docente en dos carreras | `c1.agregarDocente(d1); c2.agregarDocente(d1);` | Un mismo docente queda duplicado/asignado en múltiples carreras sin ningún tipo de control. | `En Carrera.agregarDocente:`<br>`if (!docentes.contains(docente)) {`<br>`  docentes.add(docente);`<br>`} else {`<br>`  System.out.println("Docente ya registrado");`<br>`}` |
| Composición no estricta | `Horario h = new Horario(f, "A-1");` | Cualquier clase externa puede crear Horarios independientemente de la Materia. | `En Horario.java:`<br>`Horario(LocalDate f, String a) {`<br>`  this.fecha = f;`<br>`  this.aula = a;`<br>`}`<br>`(Quitar el modificador 'public' del constructor)` |

## Ejercicio 7: Ampliación del Modelo — Clase Ayudante

**Selección y Justificación de Relaciones:** Para incorporar a un Ayudante que es estudiante y colabora con un docente, se analizan dos relaciones distintas:

**1. Ayudante y Estudiante (Herencia):** Un Ayudante ES UN Estudiante de la universidad, por lo que hereda todos sus atributos (código, inscripciones) y comportamientos. (Ayudante extends Estudiante).

**2. Ayudante y Docente (Asociación):** El Ayudante colabora con un Docente, pero ambos tienen ciclos de vida independientes. Ninguno destruye al otro. Por lo tanto, se establece una relación de Asociación donde el Ayudante conoce al Docente con el que colabora.

**Código de la Clase Ayudante:** Implementación Java de la clase Ayudante:

```
public class Ayudante extends Estudiante {
    // Asociación con Docente
    private Docente docenteAsignado;

    public Ayudante(String nombre, String ci, String codigo, Docente docente) {
```

```
        super(nombre, ci, codigo);
        this.docenteAsignado = docente;
    }

    public Docente getDocenteAsignado() {
        return docenteAsignado;
    }

    public void setDocenteAsignado(Docente docente) {
        this.docenteAsignado = docente;
    }

    @Override
    public String toString() {
        return "Ayudante " + getNombre() + " (Código: " + getCodigo() +
               ", Colabora con: " + (docenteAsignado != null ?
docenteAsignado.getNombre() : "Ninguno") + ")";
    }
}
```

**Representación UML de Ayudante:** Actualización del Diagrama UML:

En la imagen adjunta…